Gen-AI workflows in practice

Introduction

- Sr Applied Scientist
- ~10 yr of experience in Data Science domain
- Retail, Insurance, Bank and Tech
- ML, A/B testing, NLP, CV
- Web automation agent, Agent eval, Memory, Demonstration Learning

Why this workshop?

- AI agents now write, review and debug code in minutes
- So what's left for the people?
- A small production system, and the parts the AI can't do for you
- You'll do the work, and see where the tools stop

Agenda

- Session 1
  - Design discussions of a Production system
  - Analyze the code base with AI
  - Solve a real problem associated with the database with AI (see why humans are still needed)
- Session 2
  - Resolve a ticket end to end using multiple GenAI tools
  - Reflect on what AI means for teaching

Build a tool that lets anyone ask the company database a question in plain English. How would you design it?

Session 1

Questions

- How does your tool get from the question to the data?
- How do you understand the meaning of business metrics?
- Which model, and why?
- How to understand DB? What happens when DB updates?
- Once it's live, how do you know it's still giving right answers?
- What metrics do you use to setup alarms?

```
git clone https://github.com/UBC-MDS/ai-assisted-workflows.git
cd ai-assisted-workflows
```

Find out how askdb answers a question, and how it handles each of the questions you just discussed.

- workshop/session1/evidence/07-refund-rate-london-last-quarter.md
- Which one is right?
- Does your design cater to this?

- Read workshop/session1/README.md
- 10 questions, each asked 5 times, each with different SQL
- Write the business rules that make askdb answer the same way every time. Ask me if any clarification required on rules
- You'll be tested on questions you haven't seen
- Submit: email business_definitions.md to tomar.ankur287@gmail.com, team name in the subject

Real world problem

- A text-to-SQL tool that the main agent calls whenever an SOP step needs data
- Every customer's database is different: no shared benchmark, and no ground truth in production
- "How do you know the SQL is right?"

Real world problem - Solution

- Generate test questions from each customer's schema with an LLM, because no benchmark exists
- Rerun each question several times. Where the SQL varies, the model is guessing(small model)
- Paraphrase each question. All phrasings should give the same SQL. Rerunning tests stability; paraphrasing tests meaning
- An LLM compares the SQL versions and names the decisions that changed, e.g. "it can't decide what 'active' means".
- Write rules into the context until runs and paraphrases agree. Consistency is something you can control without ground truth.

Real world problem – Solution

- The customer confirms the rules and add their definitions. Run benchmarks
- Weekly: Use embedding models finetuned to identify similarity in queries and find sample. Compare SQL for the similar part
- Escalate the most confusing cases: rank by number of assumptions, and send the top few to the customer

- Caveat: ***Reruns and paraphrases can all agree and all be wrong, if the definition itself is wrong.***

BREAK

Session 2

Situation

- Ticket from legal

- Let's look at multiple AI tools we use in industry

Tools

- Ticket analyzer and resolver
  - Categorize – Upstream, Bug, Feature request
  - Docs analysis
  - Logs analysis
- Ticket router
- Coding tools
- Code reviewer
- Metric analyzer
- WBR generator

Reflection

Takeaways

- AI did the work we usually grade
  - Rules and code in minutes; ran and passed the checks
- What was left was judgment
  - Spotting a decision, finding its owner, asking: only people did that
- "Passes the checks" ≠ "is right"
  - All checks green; board retention still overstated
- Statistical thinking caught it
  - Survivorship bias, not code, explained the wrong number
- Grade the thinking, not just the output
  - Assess questions asked, assumptions flagged, how AI's work was checked